AI AGENT MASTERY

Prerequisites — Self-Study Guide

Everything You Need to Know Before Week 1

No coding or AI experience required

Estimated Duration: 10–15 hours of self-study

Complete this guide before starting Week 1

Version 1.0 — March 2026

How to Use This Guide

This self-study guide covers everything you need to know before starting Week 1 of the AI Agent Mastery curriculum. Each section includes visual diagrams, simple explanations, and hands-on examples you can try immediately.

Study Tips for Beginners

1. Go at your own pace — there is no rush. Understanding beats speed.
2. Type every code example yourself (don't copy-paste). Muscle memory helps you learn.
3. If something confuses you, re-read it once, then try the example. Code makes concepts click.
4. Use Google Colab (free) — no installation needed. Just open colab.research.google.com.
5. Estimated total: 10–15 hours. Spread across 3–5 days for best retention.

#	Module	Time	Difficulty
1	What is AI?	45 min	Concept
2	Generative AI vs. Traditional ML	45 min	Concept
3	How Large Language Models Work	1 hour	Concept
4	Python: Variables & Data Types	1 hour	Hands-On
5	Python: Conditionals & Loops	1–1.5 hrs	Hands-On
6	Python: Functions	1 hour	Hands-On
7	Python: Libraries & Imports	45 min	Hands-On
8	Making API Calls	1 hour	Hands-On
9	Prompt Engineering Fundamentals	1 hour	Concept + Lab
10	Function Calling in LLMs	1 hour	Concept + Lab

Module 1: What is AI?

Duration: 45 minutes | **Difficulty:** Beginner (no coding)

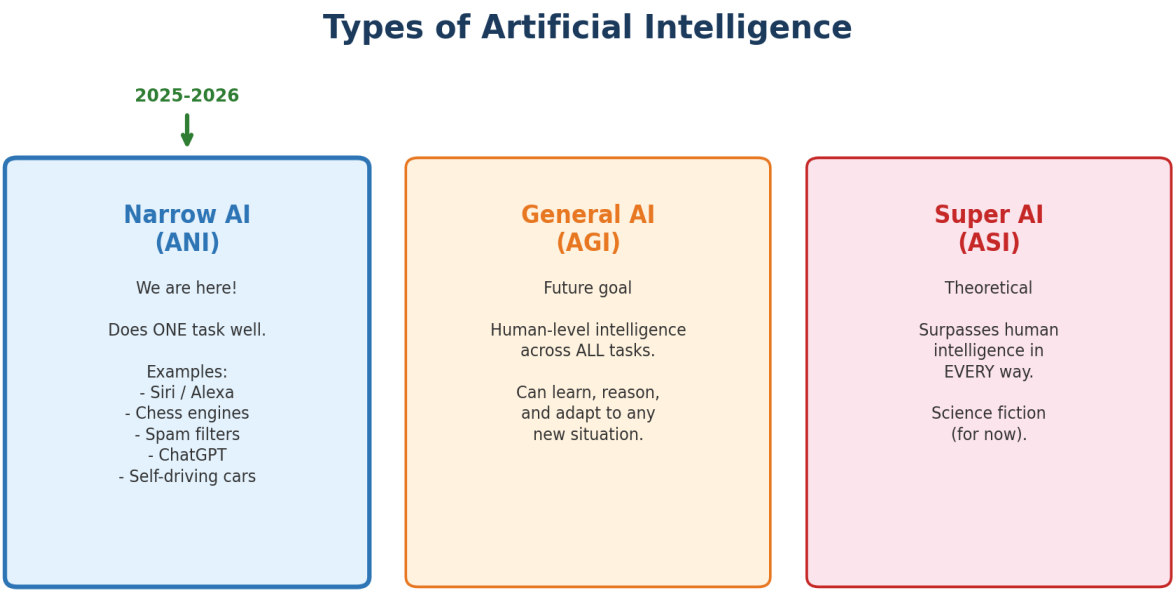
1.1 AI in Simple Words

Artificial Intelligence (AI) is when we teach computers to do things that normally require human intelligence — like understanding language, recognizing images, making decisions, or playing games.

Think of it this way: a calculator follows exact rules (2 + 2 = 4). AI goes further — it learns from examples and can handle situations it has never seen before. Show an AI 10,000 photos of cats and dogs, and it learns to tell them apart — even for photos it has never seen.

1.2 The Three Types of AI

AI exists in three categories, but only one exists today:



Where are we today? We are in the Narrow AI era. Every AI product you use — Siri, Google Search, ChatGPT, Netflix recommendations — is Narrow AI. It's incredibly powerful at specific tasks, but it cannot do things outside its training. ChatGPT can write essays but cannot drive a car.

1.3 AI You Already Use Every Day

You interact with AI more than you realize:

- **Google Search:** AI understands your query and ranks billions of pages to find the best answer.
- **Netflix / YouTube:** AI learns what you watch and recommends similar content.
- **Email Spam Filter:** AI automatically moves junk email out of your inbox.
- **Phone Camera:** AI enhances your photos (portrait mode, night mode, face detection).
- **Voice Assistants:** Siri, Alexa, Google Assistant all use AI to understand your voice.
- **Autocomplete:** When your phone predicts the next word you're typing — that's AI.

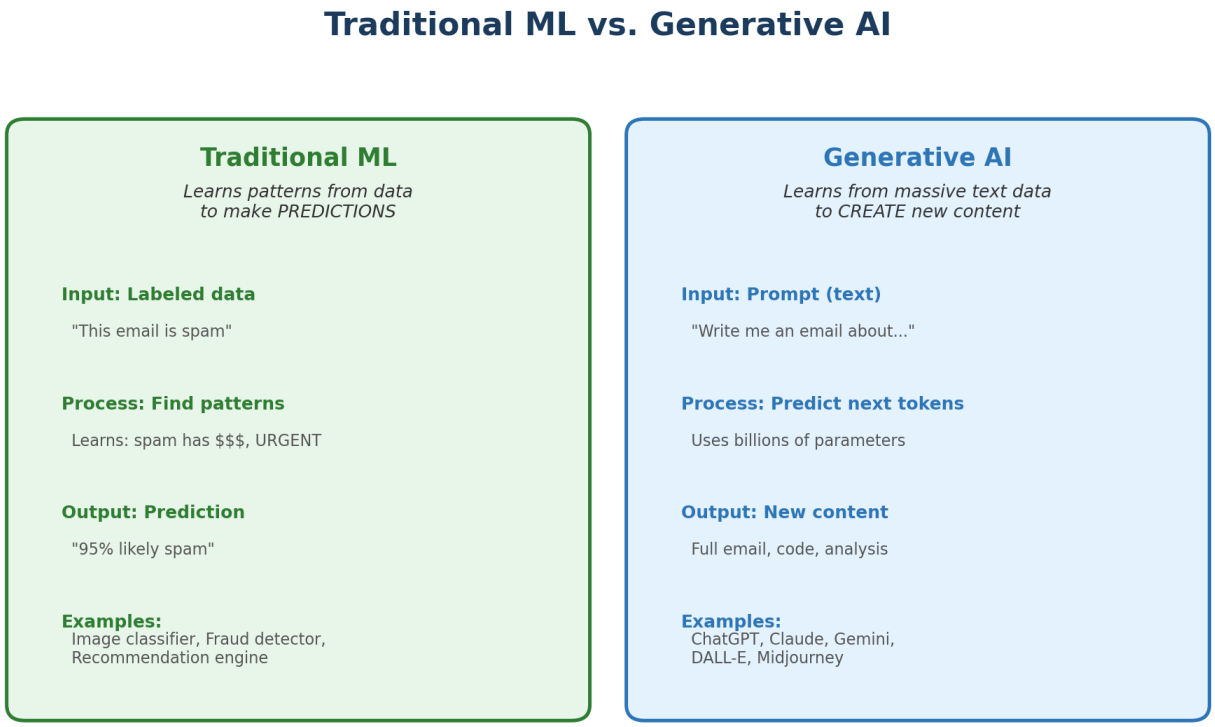
> **Key Takeaway**
> AI is not magic or science fiction. It's math + data + computing power. In this course, you'll learn to build AI agents — programs that use AI to autonomously accomplish tasks, like researching a topic or answering customer questions.

Module 2: Generative AI vs. Traditional ML

Duration: 45 minutes | **Difficulty:** Beginner (no coding)

2.1 Two Branches of AI

Machine Learning (ML) is a subset of AI. There are two main types you need to understand:



2.2 Traditional ML — The Predictor

Traditional ML learns patterns from labeled data to make predictions. You show it thousands of examples with correct answers, and it learns the pattern.

Real-World Analogy: Imagine teaching a child to identify fruits. You show them 100 apples saying ‘this is an apple’ and 100 bananas saying ‘this is a banana.’ Eventually, they can identify new fruits they’ve never seen before. That’s Traditional ML.

Examples of Traditional ML:

- **Email Spam Detection:** Trained on millions of emails labeled ‘spam’ or ‘not spam.’ Predicts new emails.

- **Credit Card Fraud:** Trained on transaction history. Flags unusual purchases.
- **Product Recommendations:** Amazon's 'Customers also bought' — trained on purchase patterns.
- **Medical Diagnosis:** Trained on X-rays labeled by doctors. Spots tumors in new scans.

2.3 Generative AI — The Creator

Generative AI doesn't just predict — it creates entirely new content: text, images, code, music, and more. It learned from vast amounts of data and can produce things that never existed before.

Real-World Analogy: A Traditional ML student studies all past exam answers to predict what the next answer will be. A Generative AI student reads every book in the library and can write a completely new book on any topic you ask for.

Examples of Generative AI:

- **ChatGPT / Claude / Gemini:** Generate human-quality text from prompts.
- **DALL-E / Midjourney:** Create images from text descriptions.
- **GitHub Copilot:** Generates working code from comments and descriptions.
- **Music AI (Suno, Udio):** Creates original songs in any genre from a text prompt.

Why This Matters for Agents

AI Agents are built on top of Generative AI (specifically, Large Language Models like ChatGPT and Claude). The LLM is the 'brain' of the agent — it reasons, plans, and decides. You are essentially giving a Generative AI model the ability to take actions in the real world.

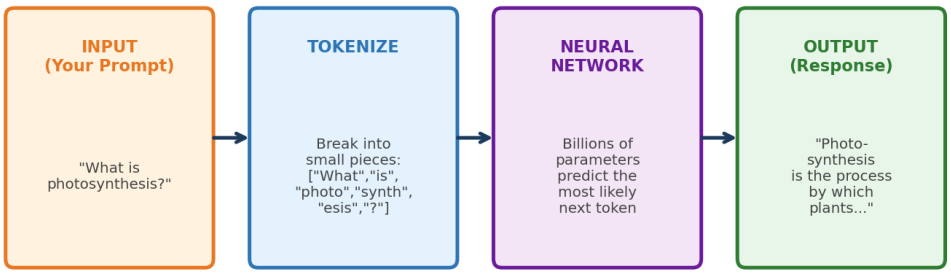
Module 3: How Large Language Models Work

Duration: 1 hour | **Difficulty:** Beginner (no coding)

3.1 What is a Large Language Model?

A Large Language Model (LLM) is an AI system trained on enormous amounts of text (books, websites, code, articles) to understand and generate human-like language. Think of it as the most well-read entity on Earth — it has read more text than any human could in a lifetime.

How Large Language Models (LLMs) Work — Simplified



Key Insight: LLMs don't "understand" — they predict the most probable next word based on patterns learned from billions of text examples.

3.2 How Does an LLM Actually Work?

At its core, an LLM does one simple thing: it predicts the next word (or 'token'). That's it. But it does this so well that the result looks like genuine understanding.

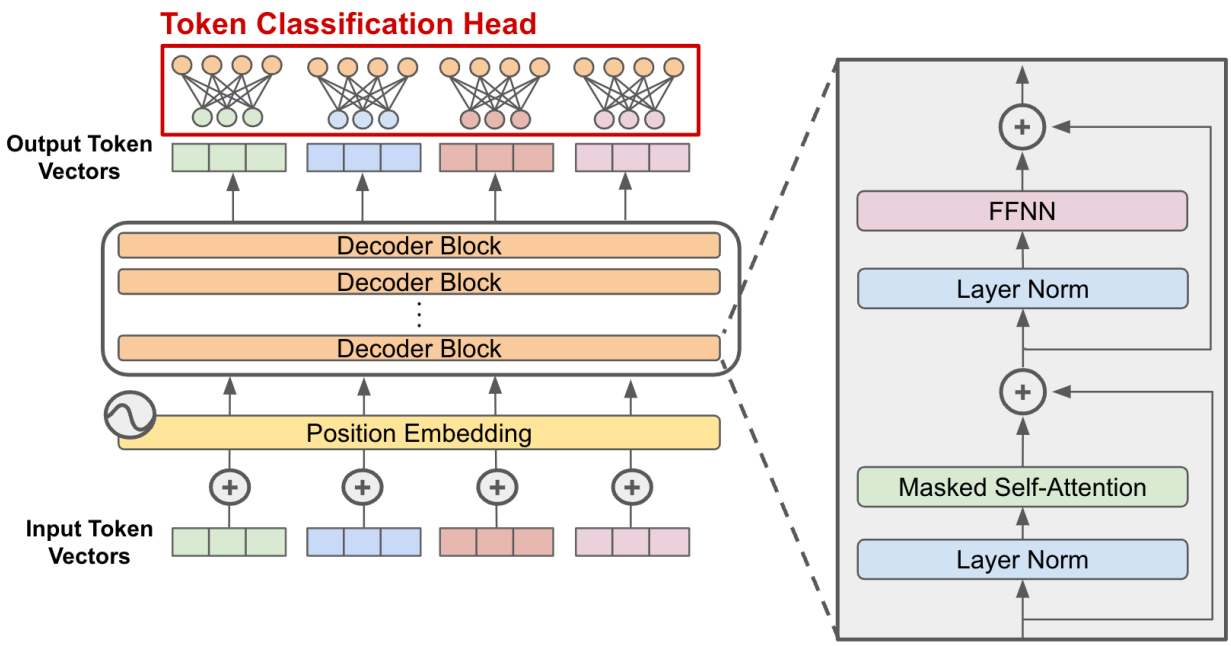
Example: If you type 'The capital of France is', the LLM calculates that the most likely next word is 'Paris' (99.7% probability), followed by a period. It generates one token at a time, each conditioned on everything before it.

Key Concepts Simplified:

- **Tokens:** LLMs don't read words — they read tokens. A token is roughly 3/4 of a word. 'Photosynthesis' becomes [Photo, synth, esis] = 3 tokens. 'I am happy' = 3 tokens. This matters because LLMs charge per token.

- **Parameters:** These are the 'knobs' the model learned during training. GPT-4 has ~1.8 trillion parameters. More parameters generally means more capability (and more cost).
- **Context Window:** The maximum text the LLM can 'see' at once. GPT-4o can see ~128,000 tokens (roughly 300 pages). Claude 3 can see ~200,000 tokens. This is the LLM's working memory.
- **Temperature:** Controls randomness. Temperature 0 = always picks the most likely word (deterministic). Temperature 1 = more creative/random. For agents, lower temperature (0–0.3) is usually better because you want reliable, consistent behavior.

Architecture



Large Language Models explained briefly
https://www.youtube.com/watch?v=LPZh9BOjkQs

Tokenizer:
https://tiktokenizer.vercel.app/?model=cl100k_base

Transformer visualisation:
https://poloclub.github.io/transformer-explainer/

llm visualizer:
https://bbycroft.net/llm

Deep Dive into LLMs like ChatGPT
https://www.youtube.com/watch?v=7xTGNNLPyMI

3.3 Popular LLMs You'll Use in This Course

Model	Company	Best For	Free?
GPT-4o / 4o-mini	OpenAI	General purpose, coding	Mini is very cheap
Claude 3.5 Sonnet	Anthropic	Complex reasoning, safety	Free tier available
Gemini 3.0 Flash	Google	Fast, multimodal, free	Generous free tier
Llama 3	Meta	Open-source, self-hosted	Completely free

Remember

LLMs don't actually 'understand' anything. They are extremely sophisticated pattern matchers. When ChatGPT explains quantum physics, it's not 'thinking' — it's predicting what a knowledgeable response would look like based on its training data. This distinction matters when building agents, because agents can hallucinate (make up facts with confidence).

Module 4: Python Basics — Variables & Data Types

Duration: 1 hour | **Difficulty:** Hands-On | Open **colab.research.google.com** and follow along

4.1 What is a Variable?

A variable is a named container that stores a value. Think of it as a labeled box: you give the box a name, and put something inside it.

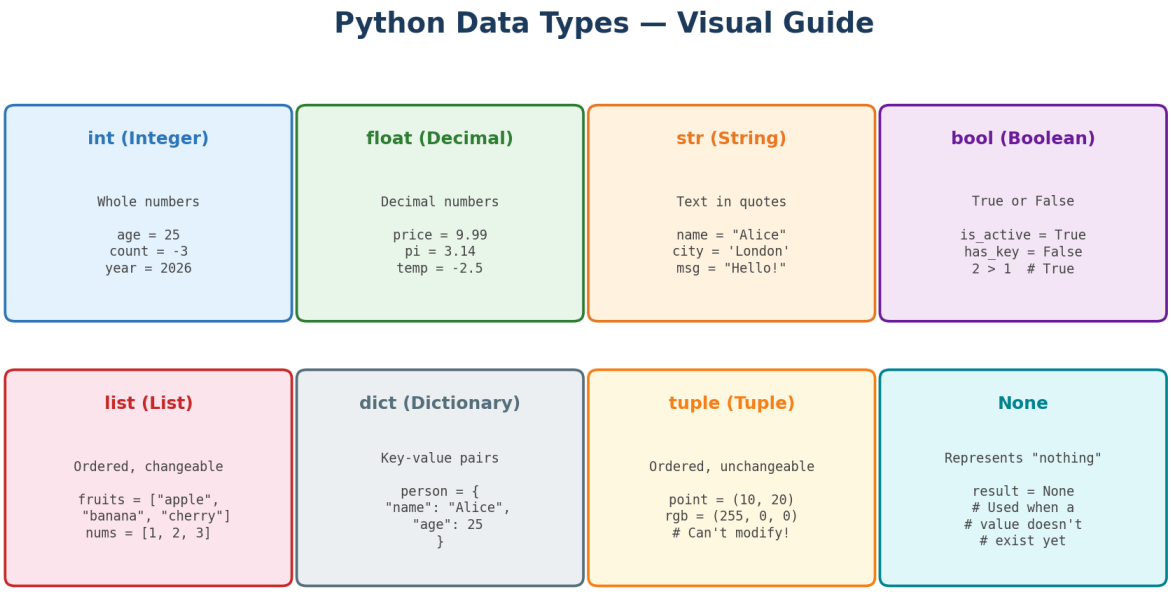
Your First Variables (Type this in Colab!)

```
# Variables are created with the = sign
name = "Alice"        # A text value (string)
age = 25              # A whole number (integer)
height = 5.6          # A decimal number (float)
is_student = True     # A true/false value (boolean)

# Print them to see the values
print(f"Name: {name}")
print(f"Age: {age}")
print(f"Height: {height} feet")
print(f"Is student? {is_student}")

# Output:
# Name: Alice
# Age: 25
# Height: 5.6 feet
# Is student? True
```

4.2 Python Data Types — Visual Guide



4.3 Working with Lists and Dictionaries

Lists and dictionaries are the two most important data structures for AI agents. Agents use lists to store messages and dictionaries to store state.

Lists — Ordered Collections

```
# Creating a list
fruits = ["apple", "banana", "cherry"]

# Access by index (starts at 0!)
print(fruits[0])    # apple
print(fruits[1])    # banana
print(fruits[-1])   # cherry (last item)

# Add and remove items
fruits.append("mango")      # Add to end
fruits.remove("banana")     # Remove by value
print(fruits)               # ['apple', 'cherry', 'mango']

# Length
print(len(fruits))          # 3
```

Dictionaries — Key-Value Pairs (Agent State!)

```
# Dictionaries store data with named keys
# This is EXACTLY how agent state works!
agent_state = {
    "user_query": "What is AI?",
```

```
    "messages": [],
    "is_complete": False,
    "iteration": 0
}

# Access values by key
print(agent_state["user_query"])  # What is AI?

# Update values
agent_state["iteration"] = 1
agent_state["messages"].append("AI is...")

# Check if key exists
if "user_query" in agent_state:
    print("Query found!")
```

Why This Matters for Agents

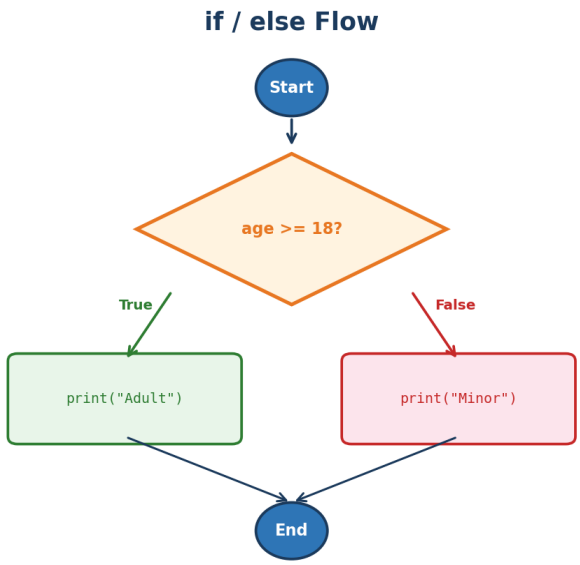
Agent state is stored as a Python dictionary. Every agent you build will have a state dictionary with keys like ‘messages’, ‘query’, ‘results’. Mastering dictionaries now saves you hours of confusion later.

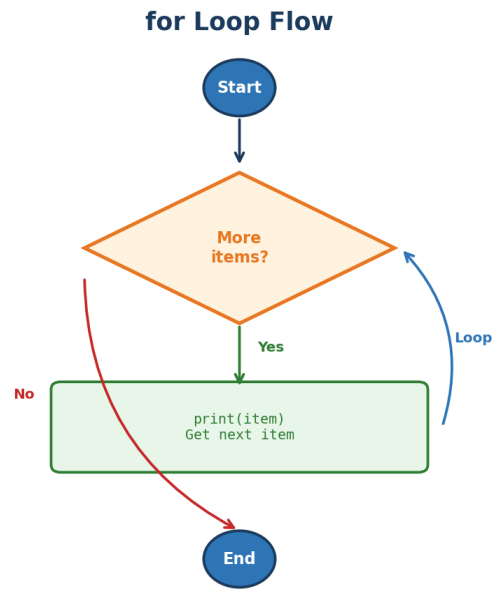
Module 5: Python Basics — Conditionals & Loops

Duration: 1–1.5 hours | **Difficulty:** Hands-On

5.1 Conditionals (if / elif / else)

Conditionals let your code make decisions. This is exactly how agents decide what to do next: 'If the user asked a question, search for an answer. Else if they said goodbye, end the conversation.'





Conditionals in Action

```
# Simple if/else
age = 20

if age >= 18:
    print("You are an adult")
elif age >= 13:
    print("You are a teenager")
else:
    print("You are a child")

# Agent decision example:
agent_action = "tool_call"

if agent_action == "tool_call":
    print("Executing tool...")
elif agent_action == "final_answer":
    print("Returning answer to user")
```

```
elif agent_action == "ask_human":
    print("Asking for clarification")
else:
    print("Unknown action!")
```

5.2 Loops (for and while)

Loops repeat code. Agents use loops to iterate through search results, process messages, and run their main observe-think-act cycle.

For Loop — Iterate Over Items

```
# Loop through a list
cities = ["London", "Tokyo", "Paris"]
for city in cities:
    print(f"Checking weather in {city}...")

# Loop with index
for i, city in enumerate(cities):
    print(f"{i+1}. {city}")
# Output: 1. London  2. Tokyo  3. Paris

# Loop through dictionary (agent state!)
state = {"query": "AI", "status": "running", "step": 3}
for key, value in state.items():
    print(f"  {key}: {value}")
```

While Loop — The Agent Loop!

```
# This is the fundamental pattern of every AI agent!
max_iterations = 5
iteration = 0
task_complete = False

while not task_complete and iteration < max_iterations:
    iteration += 1
    print(f"Agent loop iteration {iteration}...")

    # Simulate agent deciding if it's done
    if iteration >= 3:
        task_complete = True
        print("Task complete!")

if not task_complete:
    print("WARNING: Max iterations reached!")
```

The Agent Connection

Every AI agent is essentially a while loop:

```
while not done and iteration < max_limit:
    observe()   # Get input / tool results
    think()     # LLM decides next action
    act()       # Execute tool or return answer
```

The loop continues until the agent decides it's finished or hits the safety limit.

Module 6: Python Basics — Functions

Duration: 1 hour | **Difficulty:** Hands-On

6.1 What is a Function?

A function is a reusable block of code that performs a specific task. You give it a name, optionally pass in data (parameters), and it returns a result. Functions are how you define agent tools.

Your First Function

```
# Define a function with 'def'
def greet(name):
    """Greet someone by name."""
    return f"Hello, {name}! Welcome to AI Agent Mastery."

# Call the function
message = greet("Alice")
print(message)  # Hello, Alice! Welcome to AI Agent Mastery.

# Functions can have default parameters
def greet_with_title(name, title="student"):
    return f"Hello {title} {name}!"

print(greet_with_title("Bob"))           # Hello student Bob!
print(greet_with_title("Bob", "Dr."))    # Hello Dr. Bob!
```

6.2 Functions as Agent Tools

In AI agents, every tool is simply a Python function. The LLM reads the function's name and description (the docstring), decides whether to call it, and passes the right arguments.

A Function That Could Be an Agent Tool

```
def calculate_bmi(weight_kg: float, height_m: float) -> str:
    """Calculate Body Mass Index and return a health assessment.

    Args:
        weight_kg: Weight in kilograms
        height_m: Height in meters

    Returns:
        A string with BMI value and category
    """
    bmi = weight_kg / (height_m ** 2)

    if bmi < 18.5:
```

```
        category = "Underweight"
    elif bmi < 25:
        category = "Normal weight"
    elif bmi < 30:
        category = "Overweight"
    else:
        category = "Obese"

    return f"BMI: {bmi:.1f} - Category: {category}"

# Use it
print(calculate_bmi(70, 1.75))  # BMI: 22.9 - Category: Normal weight

# When this becomes an agent tool, the LLM reads the
# docstring to understand what the function does,
# then passes weight_kg and height_m automatically!
```

Functions + Type Hints = Better Tools

Notice the ‘: float’ and ‘-> str’ annotations? These are type hints. They tell the LLM (and other developers) exactly what type of data the function expects and returns. Always add type hints to your agent tool functions — they dramatically improve the LLM’s ability to use your tools correctly.

Module 7: Python Basics — Libraries & Imports

Duration: 45 minutes | **Difficulty:** Hands-On

7.1 What Are Libraries?

Libraries are pre-written code that other developers have created and shared. Instead of writing everything from scratch, you import a library and use its functions. This is like using pre-built LEGO pieces instead of making each brick yourself.

Importing Libraries

```
# Import an entire library
import math
print(math.sqrt(16))     # 4.0
print(math.pi)           # 3.141592653589793

# Import specific items
from datetime import datetime
now = datetime.now()
print(f"Current time: {now}")

# Import with an alias (nickname)
import json  # We use this A LOT for APIs

# Convert Python dict to JSON string
data = {"name": "Alice", "age": 25}
json_string = json.dumps(data, indent=2)
print(json_string)

# Convert JSON string back to Python dict
parsed = json.loads(json_string)
print(parsed["name"])  # Alice
```

7.2 The ‘requests’ Library — Your Gateway to APIs

The requests library is the most important library for AI agent development. It lets you call any web API — weather data, search engines, databases, and of course, LLM APIs.

Using the requests Library

```
import requests

# GET request — retrieve data
response = requests.get("https://wttr.in/London?format=j1")

# Check if the request succeeded
if response.status_code == 200:
```

```
    data = response.json()  # Parse JSON response
    temp = data['current_condition'][0]['temp_C']
    print(f"London temperature: {temp}°C")
else:
    print(f"Error: {response.status_code}")

# POST request — send data (used for LLM API calls)
# response = requests.post(url, json={...}, headers={...})
```

7.3 Key Libraries for This Course

Library	Purpose	Install Command
requests	Make HTTP API calls	pip install requests
json	Parse/create JSON data	(built-in, no install)
langchain	LLM integrations, tools	pip install langchain
langgraph	Build agent graphs	pip install langgraph
google-adk	Google's agent framework	pip install google-adk
pydantic	Data validation/schemas	pip install pydantic
arize-phoenix	Agent tracing/debugging	pip install arize-phoenix

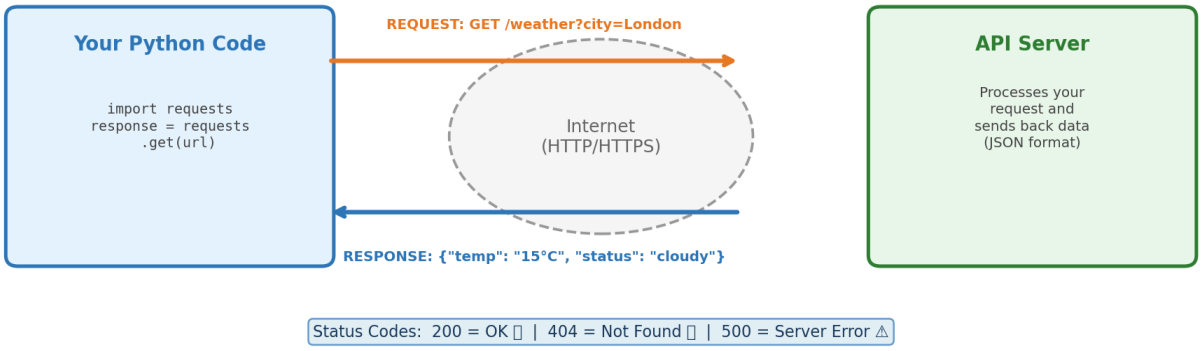
Module 8: Making API Calls

Duration: 1 hour | **Difficulty:** Hands-On

8.1 What is an API?

An API (Application Programming Interface) is a way for your code to talk to another service over the internet. When your agent calls an LLM, checks the weather, or searches the web — it's making API calls.

How API Calls Work — Request & Response



8.2 Anatomy of an API Call

Every API call has four parts:

- **URL (endpoint):** Where to send the request. Example: https://api.openai.com/v1/chat/completions
- **Method:** GET (retrieve data) or POST (send data). LLM calls are POST.
- **Headers:** Metadata like your API key. Example: Authorization: Bearer sk-abc123...
- **Body (payload):** The data you're sending. For LLMs, this is your prompt in JSON format.

```
Complete API Call Example — Calling an LLM
import requests
import json
```

```
# This is what happens UNDER THE HOOD when you use
# ChatGPT or Claude. Your agent makes calls like this:

url = "https://api.openai.com/v1/chat/completions"

headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {OPENAI_API_KEY}"
}

payload = {
    "model": "gpt-4o-mini",
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is an AI agent?"}
    ],
    "temperature": 0.7,
    "max_tokens": 200
}

response = requests.post(url, headers=headers, json=payload)
data = response.json()

# Extract the assistant's reply
answer = data["choices"][0]["message"]["content"]
print(answer)
```

8.3 Understanding JSON

JSON (JavaScript Object Notation) is the universal language of APIs. It looks exactly like a Python dictionary:

```
JSON — The Language of APIs
import json

# JSON looks like a Python dictionary
api_response = '{"name": "London", "temp": 15, "conditions": "cloudy"}'

# Parse JSON string into a Python dictionary
data = json.loads(api_response)
print(data["name"])       # London
print(data["temp"])       # 15

# Convert Python dictionary to JSON string
my_data = {"query": "AI agents", "max_results": 5}
json_string = json.dumps(my_data, indent=2)
print(json_string)
# Output:
# {
#   "query": "AI agents",
#   "max_results": 5
```

```
# }
```

Error Handling — Always Expect Failures

API calls can fail (server down, invalid key, rate limit). Always wrap calls in try/except:

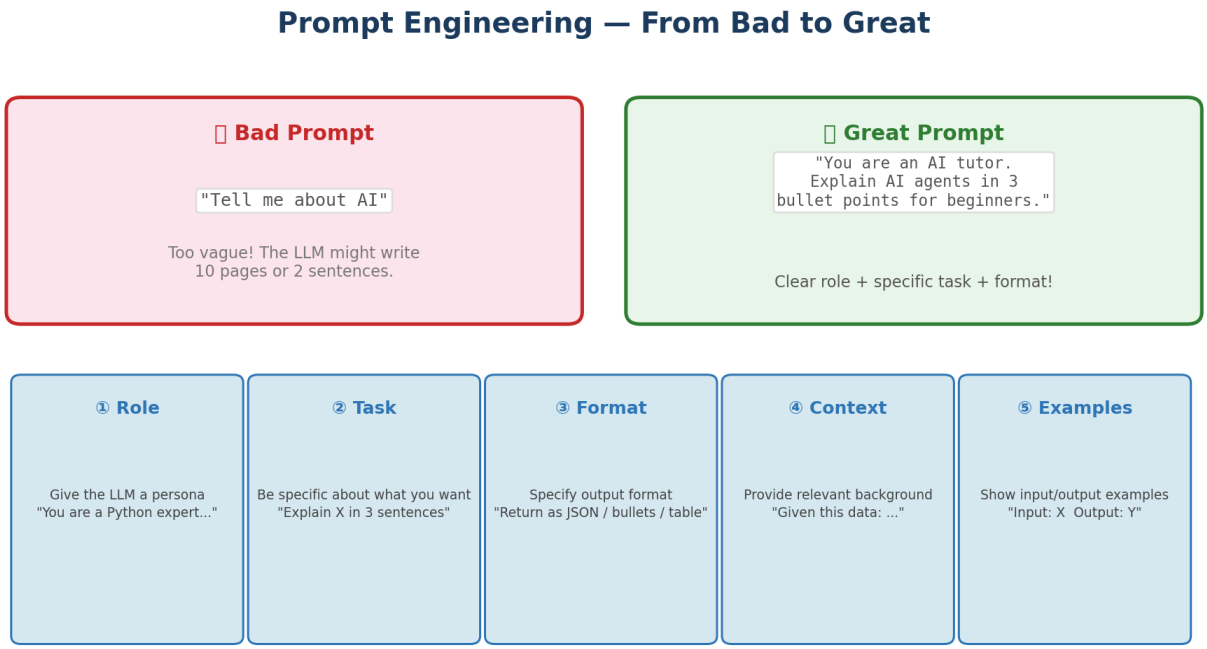
```
try:
    response = requests.get(url, timeout=10)
    response.raise_for_status()  # Raises error for 4xx/5xx
    data = response.json()
except requests.exceptions.Timeout:
    print("Request timed out — try again later")
except requests.exceptions.HTTPError as e:
    print(f"API error: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Module 9: Prompt Engineering Fundamentals

Duration: 1 hour | **Difficulty:** Concept + Practice

9.1 Why Prompts Matter

A prompt is the instruction you give to an LLM. The quality of your prompt determines the quality of the response. A vague prompt gets a vague answer. A clear, specific prompt gets a precise, useful answer. This is the single most important skill for working with AI.



9.2 The Five Principles of Great Prompts

Principle 1: Give a Role

```
Bad vs. Good — Role
# Bad: No role
"Explain machine learning"

# Good: Clear role
"You are a patient teacher explaining concepts to a 10-year-old.
Explain machine learning using a real-world analogy."
```

Principle 2: Be Specific

```
Bad vs. Good — Specificity
# Bad: Vague
"Write about Python"

# Good: Specific task + constraints
"Write a 200-word beginner-friendly explanation of Python lists.
Include 2 code examples and end with a practice exercise."
```

Principle 3: Specify the Format

```
Bad vs. Good — Format
# Bad: No format
"Compare LangGraph and ADK"

# Good: Clear format
"Compare LangGraph and ADK in a table with these columns:
Feature, LangGraph, ADK, Winner.
Include at least 5 rows."
```

Principle 4: Provide Context

```
Bad vs. Good — Context
# Bad: No context
"Is this code correct?"

# Good: Full context
"I am building a weather agent using LangGraph in Python.
This function is supposed to return temperature in Celsius,
but it returns None. Here is the code: [paste code]
What is wrong and how do I fix it?"
```

Principle 5: Give Examples (Few-Shot)

```
Few-Shot Prompting — Show What You Want
"Classify the sentiment of customer reviews.

Example 1:
Review: "This product is amazing, I love it!"
Sentiment: Positive

Example 2:
Review: "Terrible quality, broke after one day."
Sentiment: Negative

Example 3:
Review: "It works fine, nothing special."
Sentiment: Neutral
```

```
Now classify this review:
Review: "Pretty good for the price, but shipping was slow."
Sentiment: "
```

Agent Prompts Are System Prompts

When you build an agent, the main prompt is the system prompt — it defines the agent's role, capabilities, and behavior rules. A well-crafted system prompt is the difference between a useful agent and a useless one. You'll practice writing agent system prompts starting in Week 1.

Key Prompting Techniques in Agentic AI

Chain-of-Thought (CoT) Prompting: Encourages the model to break down complex tasks into intermediate, logical reasoning steps.

ReAct (Reasoning + Acting) Prompting: Combines reasoning traces (thinking) with action execution (calling tools or APIs) to solve tasks.

Prompt Chaining: Breaks a complex request into a sequence of smaller, modular prompts, where the output of one step feeds into the next.

Few-Shot Prompting: Provides a few input-output examples in the prompt to teach the agent the desired style, format, or task logic.

Role-Based/Persona Prompting: Assigns a specific persona (e.g., "You are a senior data analyst") to constrain behavior and improve task focus.

Tree of Thoughts (ToT) Prompting: Extends CoT by allowing the agent to explore multiple reasoning paths, evaluating and backtracking if necessary.

Self-Reflection/Self-Consistency Prompting: Prompts the agent to review its own output for errors and refine it.

Goal-Oriented/Task-Specific Prompting: Focused instructions providing strict objectives, constraints, and success criteria for autonomous execution.

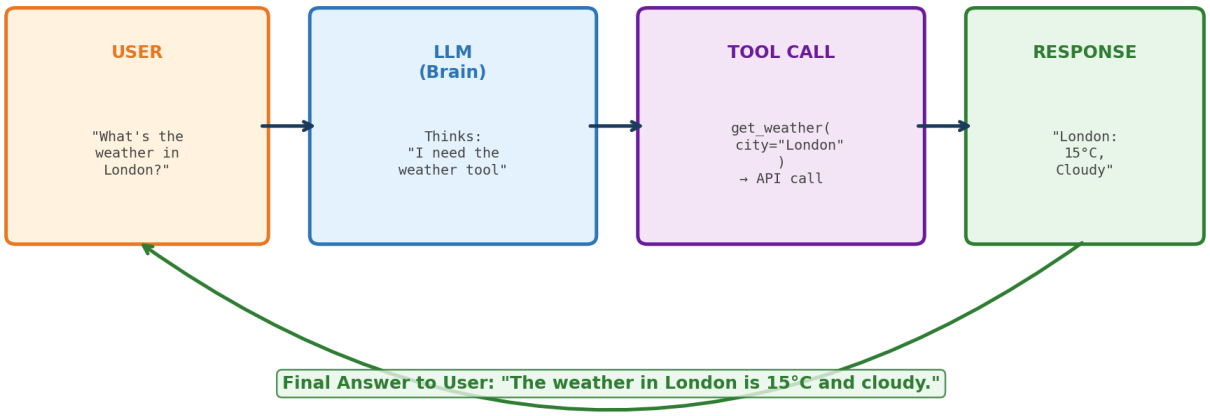
Module 10: Function Calling / Tool Use in LLMs

Duration: 1 hour | **Difficulty:** Concept + Lab

10.1 Why Function Calling is the Key to Agents

Without function calling, an LLM can only generate text. With function calling, an LLM can interact with the real world — searching the web, querying databases, sending emails, running code. This is what transforms a chatbot into an agent.

How Function Calling / Tool Use Works in LLMs



10.2 How It Works (Step by Step)

1. **Step 1 — Define Tools:** You write Python functions and describe them to the LLM (name, description, parameter schema).
2. **Step 2 — User Asks a Question:** The user says ‘What’s the weather in London?’
3. **Step 3 — LLM Decides:** The LLM reads the question and its available tools. It decides: ‘I should call get_weather with city=London.’
4. **Step 4 — Your Code Executes:** Your code receives the tool call, runs the get_weather function, and gets the actual weather data.
5. **Step 5 — LLM Synthesizes:** The LLM receives the tool result and generates a natural language answer: ‘The weather in London is 15°C and cloudy.’

10.3 Example: Defining Tools for an LLM

```
Defining Tools the LLM Can Call
# A tool is just a Python function with a clear docstring
# The LLM reads the docstring to understand what the tool does

def search_web(query: str, max_results: int = 5) -> list:
    """Search the web for information.

    Args:
        query: The search query string
        max_results: Maximum number of results to return

    Returns:
        A list of search results with title, url, and snippet
    """
    # In a real agent, this calls an actual search API
    # For now, we return mock data
    return [
        {"title": "AI Agents Explained", "url": "https://...", "snippet": "..."},
        {"title": "Building Your First Agent", "url": "https://...", "snippet":
"..."}
    ]

def calculate(expression: str) -> float:
    """Safely evaluate a mathematical expression.

    Args:
        expression: A math expression like '2 + 3 * 4'

    Returns:
        The numerical result
    """
    # Agent can now do math!
    return eval(expression)  # (In production, use a safe parser)
```

The Big Picture

Function calling is what separates an AI agent from a chatbot:

Chatbot: User question --> LLM --> Text response

Agent: User question --> LLM --> Tool call --> Real data --> LLM --> Informed response

An agent doesn't just guess the weather — it checks. It doesn't just estimate a calculation — it computes. This is why agents are so much more reliable than plain chatbots.

10.4 Try It Yourself

Practice: Write Your Own Tool Functions

```
# Exercise: Write these tool functions

# Tool 1: Convert Celsius to Fahrenheit
def celsius_to_fahrenheit(celsius: float) -> float:
    """Convert temperature from Celsius to Fahrenheit."""
    return (celsius * 9/5) + 32

# Tool 2: Get the current date and time
def get_current_time() -> str:
    """Get the current date and time as a formatted string."""
    from datetime import datetime
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")

# Tool 3: Count words in a text
def count_words(text: str) -> int:
    """Count the number of words in a text."""
    return len(text.split())

# Test them:
print(celsius_to_fahrenheit(100))    # 212.0
print(get_current_time())            # 2026-03-14 22:30:00
print(count_words("Hello world"))    # 2
```

Resources & Next Steps

Free Learning Resources

- **AI for Everyone (Coursera):** Andrew Ng's course — Modules 1–2 (2 hours)
- **Python for Everybody (Coursera):** Weeks 1–3 — variables, loops, functions (4 hours)
- **Google Colab:** Free Python notebook — no installation needed
- **Anthropic Claude 101:** Learn Claude for work tasks (~2 hours)
- **Anthropic AI Fluency:** Framework & Foundations (~2 hours)

LLM API Quickstarts

- **OpenAI:** API Quickstart (1 hour)
- **Gemini:** Getting Started (1 hour)
- **Anthropic:** Tool Use Guide (1 hour)
- **Anthropic:** Building with the Claude API (~3 hours)

Agent-Specific Primers

- **Andrew Ng:** Agentic AI Course (DeepLearning.AI) – Free intro (1 hour)
- **YouTube:** Agentic AI Design Patterns Introduction (20 min)
- **LangGraph:** Official Quickstart (1 hour)
- **Google ADK:** Installation & Setup (1 hour)

Ready for Week 1!

If you've worked through all 10 modules, you now have:

- A clear understanding of AI, Generative AI, and LLMs
- Python skills: variables, data types, conditionals, loops, functions, imports
- API calling experience with requests and JSON handling
- Prompt engineering fundamentals (role, task, format, context, examples)
- Understanding of function calling — the foundation of AI agents

You are ready to start Week 1: Agent Fundamentals and Setup!